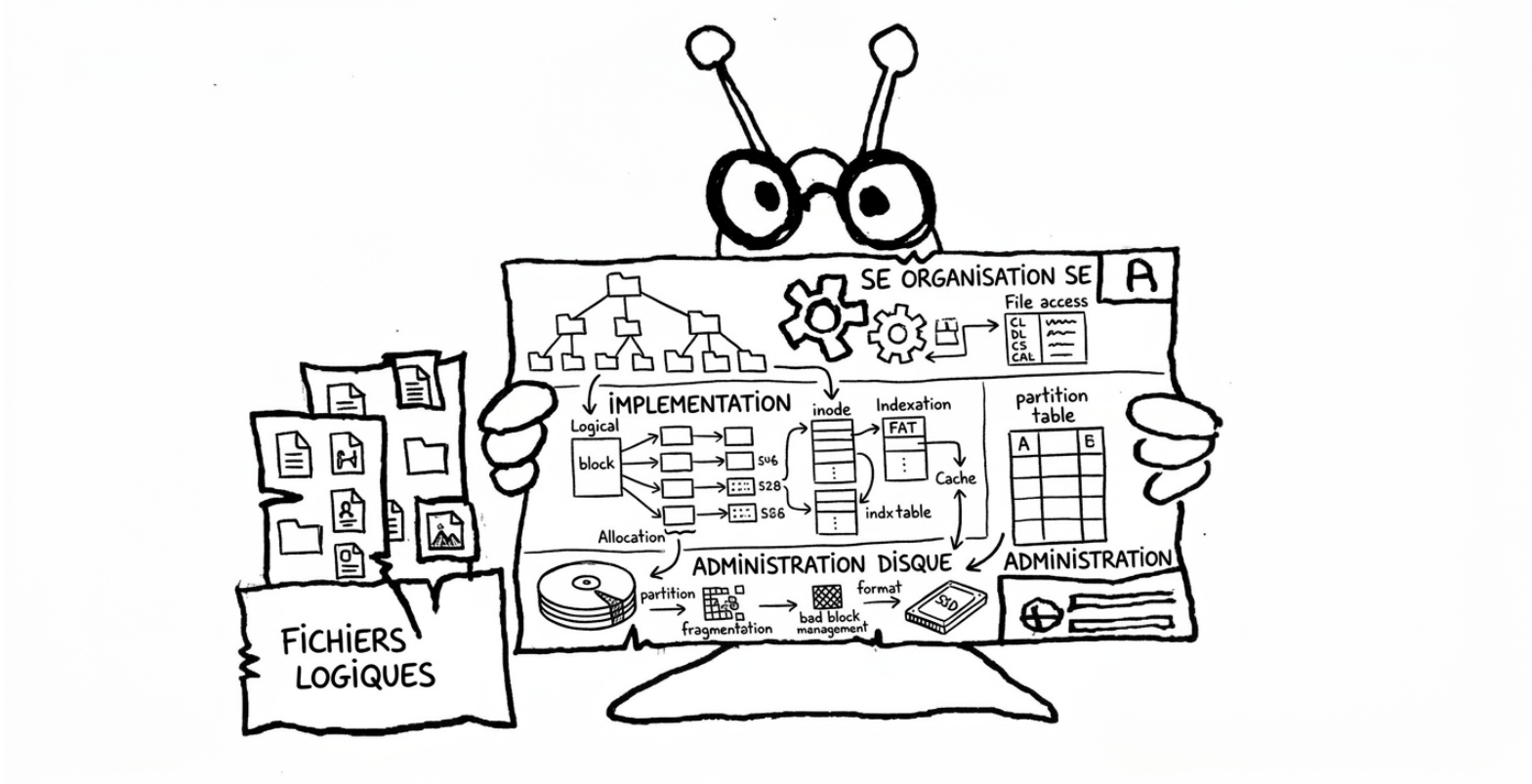
Chapitre 7: Système de Fichiers



Objectifs du chapitre

1. Décrire l'abstraction de fichier et ses attributs, opérations et types
2. Expliquer la structure de l'inode et la correspondance entre blocs logiques et physiques
3. Comparer les méthodes d'allocation contiguë, chaînée et indexée
4. Illustrer la différence entre lien physique et lien symbolique au niveau de l'inode
5. Appliquer les commandes stat, df, du, ln pour analyser un système de fichiers

Section 1 : Le concept de fichier

Définition → attributs → horodatages → opérations → table des fichiers ouverts → types

Définition : le fichier



Fichier : collection nommée et persistante de données, gérée par le SE comme une unité logique. C'est l'unité de stockage logique que l'utilisateur manipule ; le disque, lui, ne connaît que des blocs.

- Un fichier survit à la fin du processus qui l'a créé (persistance).
- Il est identifié par un nom, utilisé par les programmes pour demander lecture ou écriture.
- Le SE cache la localisation physique : l'utilisateur ne sait pas où sont les blocs sur le disque.

Tout est un fichier



Le problème

- Clavier, écran, disque, réseau : des matériels très différents
- Sans interface commune : un appel système différent pour chacun

L'idée Unix

- Un « fichier » = tout ce qu'on peut lire ou écrire comme une suite d'octets
- Donc les mêmes appels **open**, **read**, **write**, **close** marchent pour tout

Exemples:

- **Un fichier sur disque** (contient des octets) => c'est un fichier
- **Le clavier** (produit des octets (on ne peut que lire)) => c'est un fichier
- **L'écran / terminal** (reçoit des octets (on ne peut qu'écrire)) => c'est un fichier
- **Un tube (pipe)**, **un socket réseau** (transporte des octets entre programmes) => c'est un fichier

Principe de conception



Pour le **système**, un fichier, c’est tout ce qui produit ou reçoit des **octets**, et on y accède toujours de la même façon.

Les attributs d'un fichier



Chaque fichier est accompagné d'un ensemble de métadonnées que le SE maintient à jour:

- **Nom** : seul attribut réellement lisible par l'utilisateur
- **Identifiant** : numéro unique du fichier dans le système de fichiers
- **Type** : nécessaire aux systèmes qui reconnaissent des formats particuliers
- **Localisation** : pointeur vers le périphérique et la position des blocs sur le disque
- **Taille** : taille courante en octets (ou en blocs)
- **Protection** : droits d'accès qui déterminent qui peut lire, écrire ou exécuter

Les trois horodatages : atime, mtime, ctime



- **atime** (last access time) : date du dernier accès en lecture
- **mtime** (modification time) : date de la dernière modification du contenu
- **ctime** (change time) : date du dernier changement de métadonnées (permissions, propriétaire, liens)
- `ls -l` affiche par défaut **mtime** ; `ls -lu` affiche atime, `ls -lc` affiche ctime
- Modifier le **contenu** met à jour **mtime** et **ctime** ; un simple `chmod` ne touche que **ctime**
- Un fichier copié avec `cp` reçoit de nouveaux horodatages ; `cp -p` les préserve

Les opérations sur les fichiers



Le système d'exploitation expose ces actions sous forme **d'appels système**, et il y a sept opérations de base:

- *Créer*
- *Ouvrir*
- *Lire*
- *écrire*
- *Se positionner (seek)*
- *Fermer*
- *Supprimer*

Tout le reste (renommer, copier, ajouter à la fin) se construit à partir d'elles.

Le descripteur de fichier (fd)



Définition:

- **fd** est un simple entier **(0, 1, 2, 3…)**
- **Il identifie un fichier ouvert** : c'est un index dans la table des fichiers ouverts du processus
- Il est retourné par ***open()*** et passé à toutes les autres opérations (***read***, ***write***, ***lseek***, ***close***)

```
int fd = open("notes.txt", O_RDWR);          // fd = 3
write(fd, "Hello", 5);                       // « écris dans le fichier n°3 »
```

Rappel: stdin, stdout, stderr



Tout est un fichier : les descripteurs **0, 1 et 2**. Trois descripteurs déjà ouverts par le système

fd	Nom	Rôle	Connecté par défaut à
0	stdin	Entrée standard (le programme lit)	Clavier
1	stdout	Sortie standard (le programme affiche)	Écran
2	stderr	Messages d'erreur	Écran

```
./programme > sortie.txt      # fd 1 → fichier
./programme 2> erreurs.txt    # fd 2 → fichier
./programme < entree.txt      # fd 0 ← fichier
```

Table des fichiers ouverts



- Les descripteurs de fichier (0, 1, 2, ...) pointent vers les fichiers ouverts par ce processus dans un **table par processus**
- **Un table système** regroupe tous les fichiers ouverts du système, avec position courante et mode d'accès

0 1 2	stdin → clavier stdout → écran stderr → écran
3 4	notes.txt data.csv

← ouverts automatiquement

← votre 1er open()
← votre 2eme open()

```
read(0, buf, 100);          // lit au clavier
write(1, "Bonjour\n", 8);   // écrit à l'écran (comme printf)
write(2, "Erreur!\n", 8);   // message d'erreur
write(fd, "Hello", 5);      // écrit dans un fichier (fd = 3)
```

Opération: Créer un fichier



Ce que fait le système (2 étapes):

- Trouver de l’espace libre dans le système de fichiers
- Ajouter une entrée dans un répertoire

```
touch notes.txt        # crée un fichier vide
```

```
int fd = creat("notes.txt", 0644);     // crée le fichier notes.txt
```

Opération: Ouvrir un fichier



Pourquoi ?

- Sans open(), chaque opération devrait fournir le nom : recherche dans le répertoire + vérification des permissions à chaque fois
- Avec open(), on le fait une seule fois et on obtient un descripteur (**fd**)
- Obligatoire avant toute opération, sauf create et delete

Ce que fait open()

- Cherche le fichier dans le répertoire
- Vérifie le mode demandé (lecture seule, lecture-écriture, ajout seul…) contre les permissions
- Copie l'entrée dans la table des fichiers ouverts et retourne le descripteur

```
int fd = open("notes.txt", O_RDWR);          // ex: fd = 3
```

Opération: Écrire dans un fichier



Principe:

- Appel système avec **le descripteur** + **les données** + **le nombre d'octets**
- Le système garde un pointeur d'écriture (accès séquentiel)
- Le pointeur est mis à jour après chaque écriture

```
write(fd, "Hello", strlen("Hello"));          // strlen = 5
write(fd, "World", strlen("World"));          // strlen = 5
```

"Hello" fait **5 octets** : la position passe de 0 à 5. La deuxième écriture commence à 5 et n'écrase donc pas la première : la position passe à 10. Le fichier contient « **HelloWorld** » et la position est à la fin.

Opération: Lire un fichier



Principe:

- **read** copie des octets du fichier vers la mémoire du programme, à partir de la position courante.
- Appel système avec **le descripteur** + **l'adresse mémoire** où placer les données + **le nombre d'octets**
- Le pointeur de lecture est mis à jour après chaque lecture

```
char buf[6];                          // une "boîte" en mémoire de 6 cases
n = read(fd, buf, sizeof(buf) - 1);   // « lis 5 octets du fichier n°3 et mets-les dans buf »
```

fd : de quel fichier on lit
buf : où mettre les octets en mémoire
sizeof(buf) - 1 = 5 : combien d'octets au maximum
n : combien d'octets ont réellement été lus

Opération: Lire un fichier

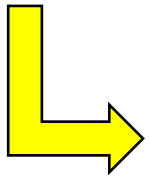


Problème:

- Pourquoi la lecture ne retourne rien ?
- Le pointeur est unique : celui de l'écriture est le même que celui de la lecture. Après avoir écrit, il est donc déjà à la fin du fichier. Si on lit tout de suite :

Position : **0 1 2 3 4 5 6 7 8 9 10**
Contenu : **H e l l o W o r l d**
Position courante : **^ (10)**

```
n = read(fd, buf, sizeof(buf) - 1);     // n = 0 → il n'y a plus rien à lire après la position 10
```



Solution: Il faut repositionner le pointeur au début

Opération: Se repositionner (seek)



Principe:

- Déplace le pointeur de position courante à une valeur donnée
- Aucune E/S réelle : seul un nombre change

```
lseek(fd, 0, SEEK_SET);      // retour au début
read(fd, buf, 5);            // lit "Hello"
lseek(fd, 5, SEEK_SET);      // saut à l'octet 5
read(fd, buf, 5);            // lit "World"
```

Position :	**0 1 2 3 4 5 6 7 8 9 10**
Contenu :	**H e l l o W o r l d**
Pointeur :	**^ (0)**
1er read :	**H e l l o**
Pointeur :	**^ (5)**
2eme read :	**W o r l d**

lseek permet un accès direct à n'importe quelle partie du fichier

Opération: Tronquer un fichier



Problème :

- Vider un fichier en gardant ses attributs (propriétaire, permissions…). Supprimer puis recréer les ferait perdre.
- **Solution** : tous les attributs restent inchangés sauf la longueur, remise à zéro, et l'espace est libéré.

```
ftruncate(fd, 0);                  // vide le fichier ouvert (fd = 3)
lseek(fd, 0, SEEK_SET);            // la position, elle, n'est pas modifiée

ls -l notes.txt                    # -rw-r--r-- 1 ilias 10 notes.txt
truncate -s 0 notes.txt
ls -l notes.txt                    # -rw-r--r-- 1 ilias 0 notes.txt
```

Usage réel : vider un fichier journal pendant qu'un programme continue d'y écrire.

Opération: Fermer un fichier



Ce que fait close():

- Le processus indique qu'il n'utilise plus le fichier
- Le système retire l'entrée de la table du processus
- Il décrémente le compteur d'ouvertures de la table système ; à zéro, l'entrée système est supprimée
- Il peut libérer les verrous détenus sur le fichier

```
close(fd);                    // fd = 3 est de nouveau libre
```

- Le prochain **open()** réutilisera le numéro 3 (plus petit numéro libre)
- Si le programme se termine, le système ferme automatiquement les fichiers restés ouverts

Opération: Supprimer un fichier



Ce que fait le système:

- Chercher le fichier nommé dans le répertoire
- Libérer tout l'espace (réutilisable par d'autres fichiers)
- Effacer ou marquer comme libre l'entrée du répertoire

```
close(fd);                          // notes.txt est fermé (fd 3 libéré)
if (unlink("notes.txt") == 0)
    printf("notes.txt supprimé\n");
else
    perror("Erreur");
```

- La commande rm notes.txt appelle **unlink()** en interne. On passe **un nom**, pas un **fd** : le fichier n'a pas besoin d'être ouvert.

Opération: Combiner les opérations de base



Exemple: copier notes.txt vers sauvegarde.txt

Procédure: Créer un fichier, puis boucler : lire l'ancien, écrire dans le nouveau

```
char buf[1024];
ssize_t n;

int src = open("notes.txt", O_RDONLY);                          // fd = 3
int dst = open("sauvegarde.txt", O_WRONLY | O_CREAT, 0644);   // fd = 4

while ((n = read(src, buf, sizeof buf)) > 0)   // lit jusqu'à fin de fichier (0)
   write(dst, buf, n);                         // écrit exactement n octets

close(src);
close(dst);
```

Opération: À retenir



- Le descripteur identifie le fichier, la position dit où l'on en est
- **Appels par nom** : **open**, **unlink**
- **Appels par descripteur** : **read, write, lseek, ftruncate, close**
- Les mêmes appels fonctionnent pour **un fichier**, **le clavier** ou **l'écran**
- Les opérations complexes (**copier, ajouter, renommer**) se construisent à partir de ces opérations de base

Les types de fichiers



- **Fichier régulier** : contient des données arbitraires, texte ou binaire
- **Répertoire** : liste d'associations nom → identifiant de fichier
- **Fichier spécial (périphérique)** : représente un matériel dans /dev, caractère ou bloc
- **Lien symbolique** : fichier dont le contenu est un chemin vers un autre fichier
- **Tube nommé (FIFO) et socket** : canaux de communication entre processus
- La commande ls -l code le type par le premier caractère : -, d, c, b, l, p, s

Pourquoi des types de fichiers, et l'extension



Le problème:

- **Un fichier** n'est qu'une **suite d'octets** : sans connaître son type, le système ne sait pas quoi en faire
- **Exemple** : afficher le code binaire d'un programme produit un texte incompréhensible
- Si le système sait que c'est un programme binaire, il peut agir de façon raisonnable
- **Technique courante**: Nom en deux parties séparées par un point : **nom.extension**

resume . docx **server . c** **ReaderThread . cpp**

- **L'extension est souvent facultative** : l'application cherche le nom donné avec l'extension qu'elle attend
- Elle n'est pas **supportée** par le système d'exploitation : **simple indice** pour les applications

Types de fichiers: Windows, macOS, UNIX

	Windows	macOS	Linux
Principe	Le type est donné par l'extension du nom	Chaque fichier a un **type** (ex. .app) et un attribut **créateur** : le programme qui l'a créé	Un **nombre magique** est stocké au **début** de certains fichiers
Qui le définit ?	L'utilisateur ou l'application, via le nom	**Le système**, pendant l'appel create()	Le contenu du fichier lui-même
Ce que le système en fait	Associe l'extension à un **programme** : .docx ouvre Word, .exe s'exécute	Lance **automatiquement** le programme créateur au double-clic	Binaire : **format des données** (ex. image) ; texte : **langage de shell** d'un script
Imposé par l'OS ?	**Oui en pratique** : l'extension détermine l'action	Oui	Non : les extensions sont de simples indices
Limite	Renommer photo.jpg en photo.txt change le programme associé, pas le contenu	Spécifique à macOS	Linux n'enregistre **pas** le programme créateur

Section 2 : Les méthodes d'accès

Accès séquentiel → accès direct → accès indexé → exemple avec lseek et dd

Accès séquentiel

Les informations sont traitées dans l'ordre, un enregistrement après l'autre.

- Pour atteindre l'octet N, il faut passer par tous les précédents
- **Opérations typiques** : *lire le suivant*, *écrire le suivant*, *rembobiner (rewind)*
- Convient aux fichiers où l'on traite la quasi-totalité du contenu
- La position courante avance automatiquement après chaque lecture ou écriture

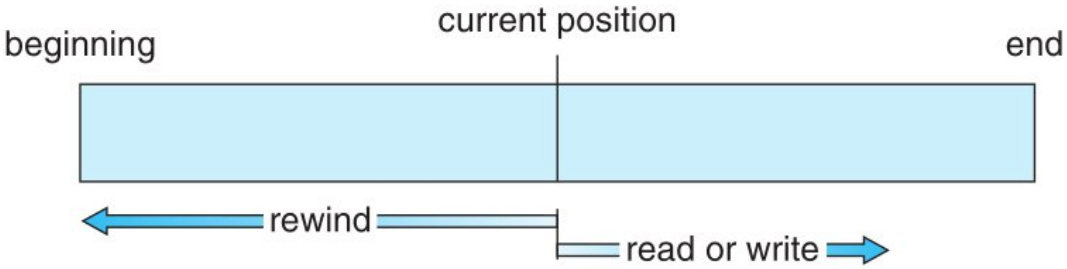


Fig 1. *Sequential-access file.*

Exemples : un éditeur de texte, un lecteur de flux, une compilation

Accès séquentiel: avantages et inconvénients

Avantages :

- Ce mécanisme d'accès aux fichiers est **simple** à mettre en œuvre.
- L'accès aux données s'effectue de manière séquentielle, **dans l'ordre où elles sont stockées**.
- **Le risque de corruption des données est plus faible**, car celles-ci sont écrites de façon séquentielle et non aléatoire.

Inconvénients :

- La **recherche** d'enregistrements spécifiques est **lente**.
- L'**insertion** ou la mise à jour d'enregistrements **au milieu** du fichier est **difficile**.
- L'espace de stockage peut être gaspillé si les enregistrements ont des longueurs variables.

Accès direct

Lecture ou écriture d'un bloc arbitraire sans parcourir les précédents

- Le fichier est vu comme une s**uite numérotée de blocs** de **taille fixe**
- L'application fournit le numéro du bloc ; le SE le traduit en position physique
- Permet de lire l'enregistrement N directement, sans lire les N−1 premiers
- Convient aux **bases de données** et aux accès periodiques à de gros fichiers

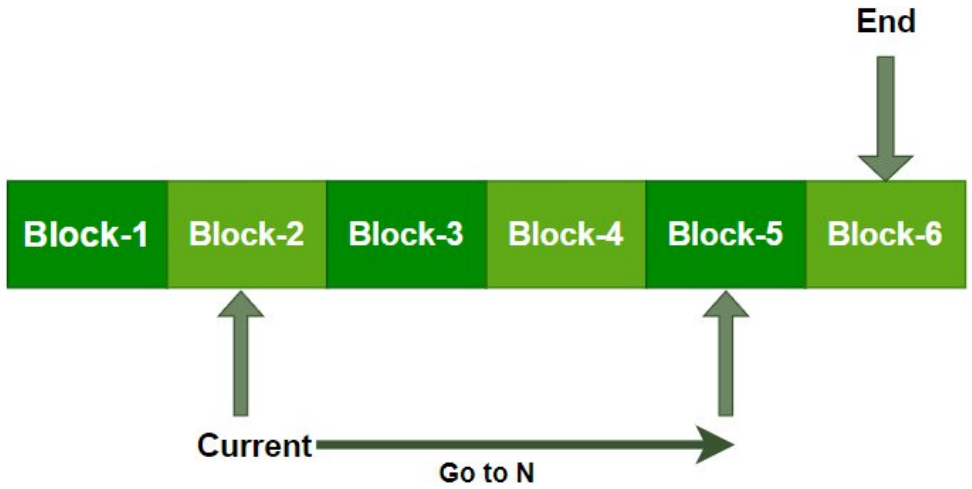


Fig 2. *Direct Access Method.*

Exemple : Dans un système de réservation de vols, toutes les informations du vol 713 sont stockées dans le bloc 713.

Accès direct: avantages et inconvénients

Avantages :

- **Les fichiers sont accessibles immédiatement**, ce qui réduit le temps d'accès moyen.
- Avec la méthode d'accès direct, il n'est pas nécessaire de parcourir tous les blocs qui précèdent celui auquel on souhaite accéder.

Inconvénients :

- **La mise en place** de l'accès direct peut s'avérer **complexe**, nécessitant des algorithmes et des structures de données sophistiqués pour gérer et localiser les enregistrements de manière efficace.
- Les méthodes d'accès direct requièrent souvent un espace de stockage supplémentaire pour conserver des informations sur la localisation des données (telles que des pointeurs ou des tables d'adresses), ce qui peut accroître les besoins globaux en matière de stockage.

Accès indexée

On retrouve un enregistrement grâce à une clé, via un index:

- Un index associe chaque clé à un pointeur vers l'enregistrement correspondant.
- Rechercher un enregistrement exige de parcourir l'index puis de sauter à la bonne position
- Le grand fichier indexé est découpé en pages pour limiter le nombre d'accès

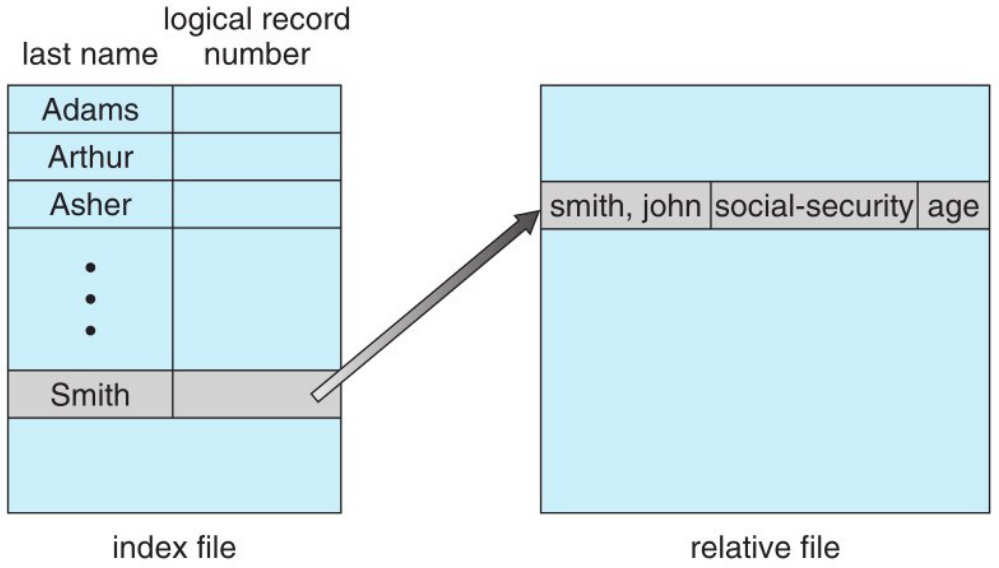


Fig 3. *Example of index and relative files.*

Exemple : un numéro de sécurité sociale retrouve directement le dossier d'un employé

Accès indexée: avantages et inconvénients

Avantages :

- **Recherche rapide** : L'index permet des recherches rapides.
- **Flexibilité** : Prend en charge aussi bien l'accès séquentiel que l'accès direct.
- **Temps d'accès réduit** : Permet de localiser rapidement des données au sein de fichiers volumineux.

Inconvénients :

- **Complexité** : Plus difficile à mettre en œuvre et à maintenir qu'un accès purement séquentiel.
- **Espace de stockage supplémentaire** : Nécessite de l'espace additionnel pour les index.
- **Mises à jour plus lentes** : Les données ainsi que les index doivent être mis à jour lors de toute insertion, suppression ou modification.

Exemple : dd pour l'accès direct

dd copie des données bloc par bloc d'une source vers une destination. C'est un outil pour illustrer l'accès direct, car il peut sauter directement à un numéro de bloc.

Syntaxe de base:

```
dd if=SOURCE of=DESTINATION bs=TAILLE_BLOC count=NOMBRE skip=N seek=M
```

Option	Signification
if=	**Input file** : le fichier ou périphérique lu (défaut : stdin)
of=	**Output file** : le fichier ou périphérique écrit (défaut : stdout)
bs=	**Block size** : taille d'un bloc en octets (ex. 512, 4K, 1M)
count=	Nombre de blocs à copier
skip=	Nombre de blocs à **sauter dans l'entrée** avant de lire
seek=	Nombre de blocs à **sauter dans la sortie** avant d'écrire
status=progress	Affiche la progression

Exemple : dd pour l'accès direct

1. **Créer le fichier de test:**

Chaque **bloc** fait **7 octets** (6 caractères + retour à la ligne).

```
printf "BLOC-0\nBLOC-1\nBLOC-2\nBLOC-3\nBLOC-4\n" > test.txt
```

2. **Lire un seul bloc avec skip**

```
dd if=test.txt bs=7 skip=3 count=1 status=none
```

3. **Lire dans le désordre (accès direct)**

```
dd if=test.txt bs=7 skip=4 count=1 status=none    # BLOC-4
dd if=test.txt bs=7 skip=1 count=1 status=none    # BLOC-1
dd if=test.txt bs=7 skip=2 count=1 status=none    # BLOC-2
```

4. **Ecrire dans un bloc précis avec seek**

```
printf "XXXXXX\n" | dd of=test.txt bs=7 seek=2 conv=notrunc status=none
```

Section 3 : Les répertoires et les chemins

Définition → opérations → structures du plat au graphe → chemins absolus et relatifs → quiz

Définition : le répertoire

Répertoire : fichier qui associe des **noms lisibles** à des **identifiants de fichiers.**

- En interne, c'est une simple table d'entrées « **nom** → **numéro** » (chez Unix, vers le numéro d'inode)
- Un répertoire peut contenir d'autres répertoires, formant une structure arborescente
- Le nom du fichier vit dans le répertoire ; le contenu et les métadonnées vivent ailleurs
- C'est cette séparation nom / données qui permet les liens multiples

RÉPERTOIRE /home/ilias

Nom	Identifiant
.	5000
..	2
notes.txt	5321
data.csv	5322
documents/	5400

Les opérations sur les répertoires

- **Rechercher un fichier** : retrouver une entrée par son nom dans le répertoire
- **Créer** : ajouter une nouvelle entrée nom → identifiant
- **Supprimer** : retirer une entrée (la suppression laisse un trou dans le répertoire)
- **Lister** : énumérer le contenu d'un répertoire (commande ls)
- **Renommer** / **déplacer** : changer le nom d'une entrée, éventuellement vers un autre répertoire
- **Traverser** : suivre un chemin en parcourant plusieurs répertoires successifs

Les opérations sur les répertoires

Rechercher et créer:

- **Exemple 1**: rechercher **notes.txt** dans **/home/ilias**

```
struct stat st;
if (stat("/home/ilias/notes.txt", &st) == 0)
    printf("Trouvé, inode = %ld\n", (long) st.st_ino);   // 5321
else
    perror("stat");                          // ex. : No such file or directory
```

On demande au système si **notes.txt** existe : si oui on affiche son numéro d'inode, sinon on affiche l'erreur.

- **Exemple 2**: créer un répertoire

```
mkdir("/home/ilias/projets", 0755);
```

Nom	Identifiant
.	5000
..	2
notes.txt	5321
data.csv	5322
documents/	5400
projets/	5601

Les opérations sur les répertoires

Supprimer et lister:

- **Exemple 1**: Supprimer une entrée

```
// retire l'entrée du fichier
unlink("/home/ilias/data.csv");

// échoue si le répertoire n'est pas vide
rmdir("/home/ilias/projets");
```

Après unlink("/home/ilias/data.csv")

Nom	Identifiant
.	5000
..	2
notes.txt	5321
(vide)	
documents/	5400
projets/	5601

- **Exemple 2**: lister **/home/ilias**

```
DIR *d = opendir("/home/ilias");          // ouvre le répertoire (un fd en interne)
struct dirent *e;
while ((e = readdir(d)) != NULL)          // une entrée à chaque appel
   printf("%s\n", e->d_name);             // ., .., notes.txt, documents...
closedir(d);                              // ferme le descripteur
```

Les opérations sur les répertoires

Renommer, déplacer et traverser:

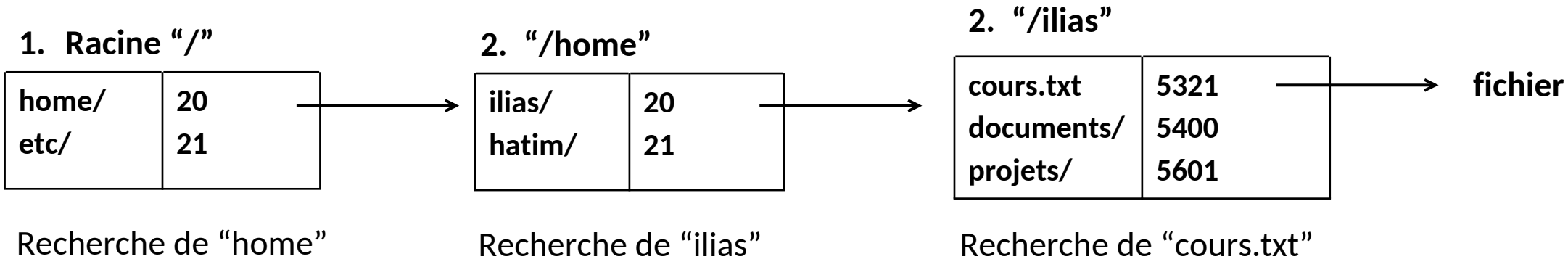
- **Exemple 1**: renommer, puis déplacer

```
// renommer:
// même identifiant 5321 : le contenu sur disque n'a pas bougé
rename("/home/ilias/notes.txt", "/home/ilias/cours.txt");
// déplacer
rename("/home/ilias/cours.txt", "/home/ilias/Documents/cours.txt");
```

Nom	Identifiant
.	5000
..	2
cours.txt	5321
documents/	5400
projets/	5601

Exemple 2: traverser /home/ilias/cours.txt

```
int fd = open("/home/ilias/cours.txt", O_RDONLY);
```



Le répertoire à un seul niveau (single-level directory)

Définition:

- C'est la structure de répertoire la plus simple
- Tous les fichiers sont contenus dans le même répertoire
- Facile à supporter et à comprendre

Limitations:

- Comme tous les fichiers sont dans le même répertoire, ils doivent avoir des noms uniques
- Avec plusieurs utilisateurs, les conflits arrivent vite

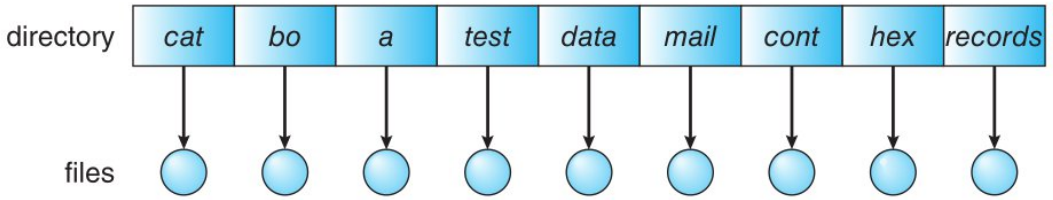


Fig 4. *Single-level directory.*

Exemple : un répertoire unique contenant **cours.txt**, **notes.txt**, **data.csv**, **rapport.txt**.
Pas de sous-dossiers, pas de chemins : le nom seul suffit pour désigner un fichier.

Le répertoire à un deux niveau (Two-Level Directory)

Principe:

- Un répertoire séparé pour chaque utilisateur (MFD)
- Créer un répertoire distinct pour chaque utilisateu
- Quand un utilisateur désigne un fichier, seul son propre UFD est cherché

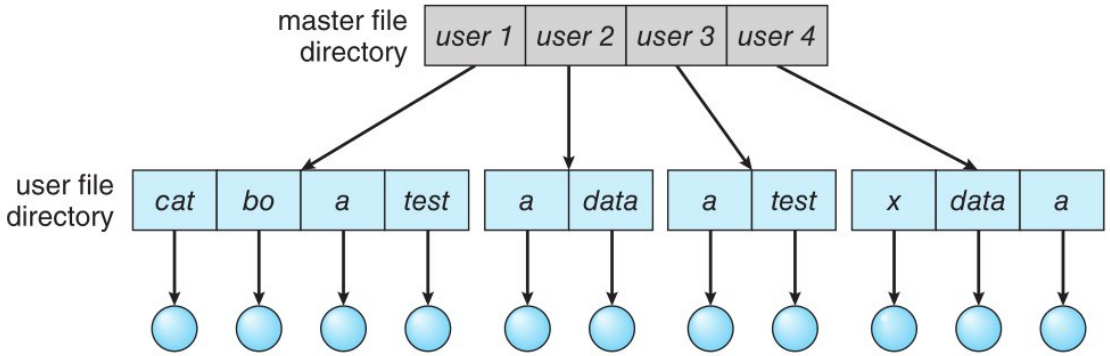


Fig 5. *Two-level directory.*

Avantage:

- La structure isole les utilisateurs les uns des autres
- Avantage si les utilisateurs sont totalement indépendants

Inconvénient s'ils veulent coopérer et accéder aux fichiers des autres. Certains systèmes interdisent simplement l'accès aux fichiers locaux d'un autre utilisateur

Le répertoire à un trois niveau (Tree-Structured Directories)

Principe:

- Une généralise la structure à deux niveaux
- Les utilisateurs peuvent créer leurs propres sous-répertoires et organiser leurs fichiers
- L'arbre a un répertoire racine, et chaque fichier a un nom de chemin unique
- Un répertoire est simplement un autre fichier, traité spécialement, avec le même format interne
- **Un bit dans chaque entrée indique si c'est un fichier (0) ou un sous-répertoire (1)**
- Des appels système spéciaux créent et suppriment les répertoires : le système implémente un autre format de fichier, celui du répertoire

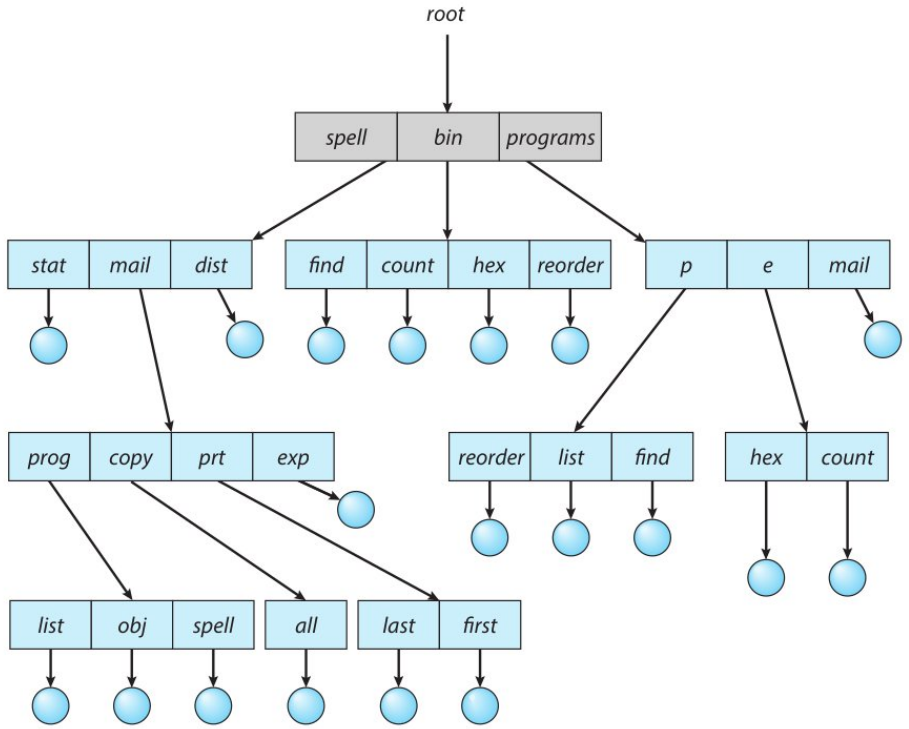


Fig 6. *Tree-Structured Directories*

Graphe acyclique

Principe:

- Dans un **arbre**, chaque fichier ou répertoire a un seul parent : **le partage est interdit**
- Dans un **graphe acyclique** (sans cycle: pas de retour), un même **fichier** ou **sous-répertoire** peut apparaître **dans plusieurs répertoires**
- **Partagé ≠ copié** : il n'existe qu'un seul fichier réel, donc toute modification est immédiatement visible par tous

- **Exemple:**
  1. Un fichier partagé : **count**
  2. Un sous-répertoire partagé : **{list, rade, w7}**
  3. Un fichier partagé dans le répertoire du bas : **all**

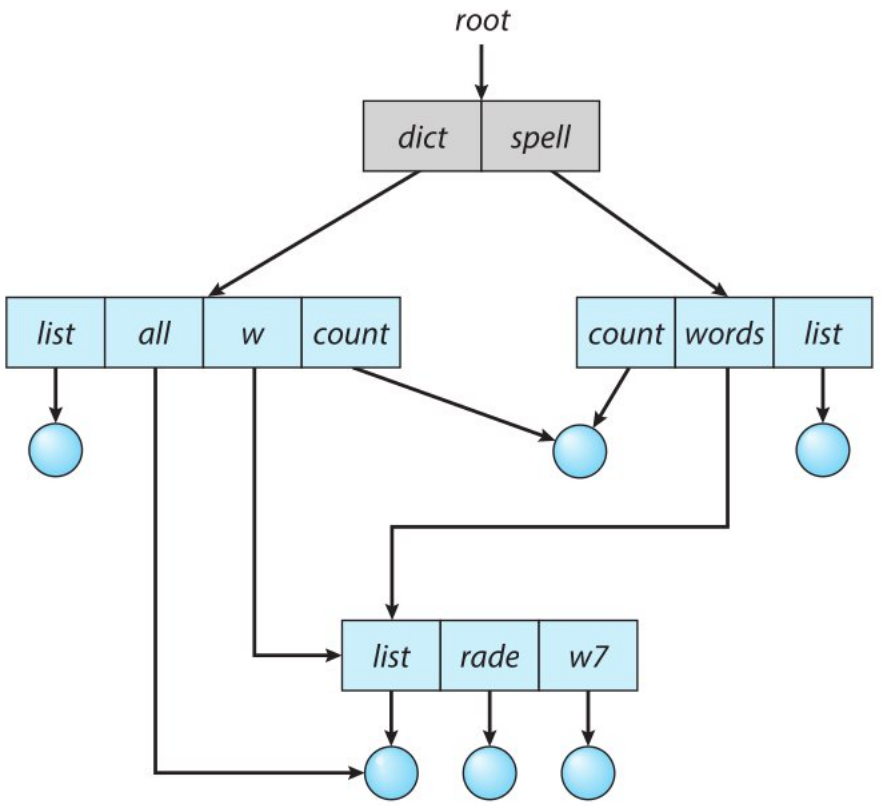


Fig 7. *Acyclic-graph directory structure.*

Chemins absolus, relatifs et répertoire courant

- **Chemin absolu** : commence à la racine /, par exemple /home/ilias/rapport.txt
- **Chemin relatif** : se résout à partir du répertoire courant, par exemple ../ilias/data.txt
- Le répertoire courant est mémorisé par processus ; . le désigne, .. désigne le parent
- Le séparateur est / sous **Unix**, \ sous **Windows** — convention différente, même idée
- **pwd** affiche le chemin absolu du répertoire courant ; cd le change
- **L'ouverture d'un fichier exige de traverser tous les répertoires du chemin**

Quiz 1 : fichier, accès et répertoires

1. **Sous Unix, un fichier est avant tout…**

   A) un ensemble d'enregistrements de taille fixe · B) une suite d'octets non structurée ·

   C) toujours du texte · D) une table en mémoire

2. **Quelle structure associe les noms de fichiers à leurs identifiants ?**

   A) l'inode · B) le superbloc · C) le répertoire · D) la table des fichiers ouverts

3. **Pour lire directement l'enregistrement n°500 d'un gros fichier, on utilise…**

   A) l'accès séquentiel · B) cat · C) l'accès direct (lseek)

4. **Un chemin relatif se résout à partir…**

   A) de la racine / · B) du répertoire courant ·

   C) de /home · D) du répertoire parent uniquement

Section 4 : Implémentation — inodes et allocation

Conception en couches → structure sur disque → superbloc → inode →
allocation contiguë/chaînée/indexée → comparaison

Conception en couches du système de fichiers

Exemple: Le trajet d'un **read** à travers les couches quand on lit **notes.txt**

le programme pense en **noms**, le disque ne comprend que des **numéros de blocs et des signaux électroniques**. Chaque couche réalise une seule traduction et ne connaît que celle du dessous.

```
// lire 100 octets de /home/ilias/notes.txt
int fd = open("notes.txt", O_RDWR);
read(fd, buf, 100);
```

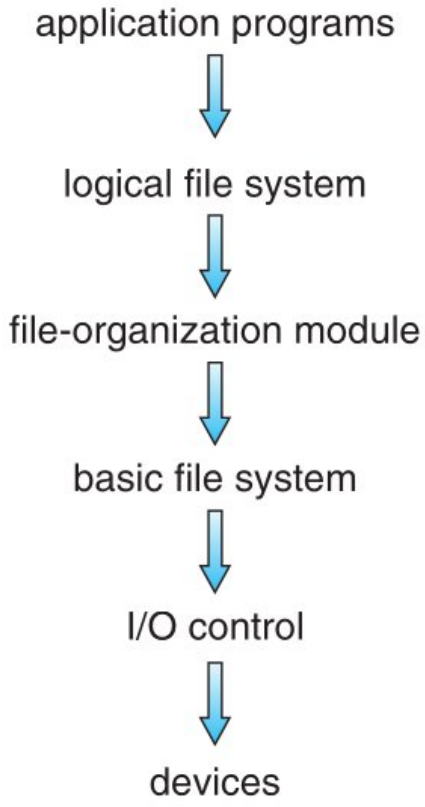


Fig 8. *Layered file system.*

Conception en couches du système de fichiers

```
// lire 100 octets de /home/ilias/notes.txt
• int fd = open("notes.txt", O_RDWR);
read(fd, buf, 100);
```

Couche 1: PROGRAMME

« donne-moi 100 octets de notes.txt »

Couche 2: SYSTÈME DE FICHIERS LOGIQUE

nom → FCB (inode 5321) : permissions OK ?

où est le contenu du fichier ?

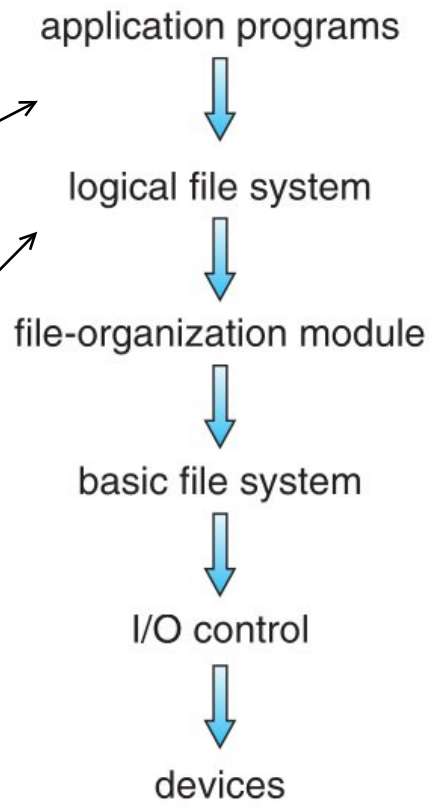


Fig 8. *Layered file system.*

Conception en couches du système de fichiers

```
// lire 100 octets de /home/ilias/notes.txt
int fd = open("notes.txt", O_RDWR);
read(fd, buf, 100);
```

Couche 3: MODULE D'ORGANISATION DES FICHIERS

octets 0 à 99 = bloc logique 0 du fichier

bloc logique 0 → bloc physique 4021

Couche 4: SYSTÈME DE FICHIERS DE BASE

« lire le bloc 4021 »

déjà dans le cache ? sinon, alloue un tampon en mémoire

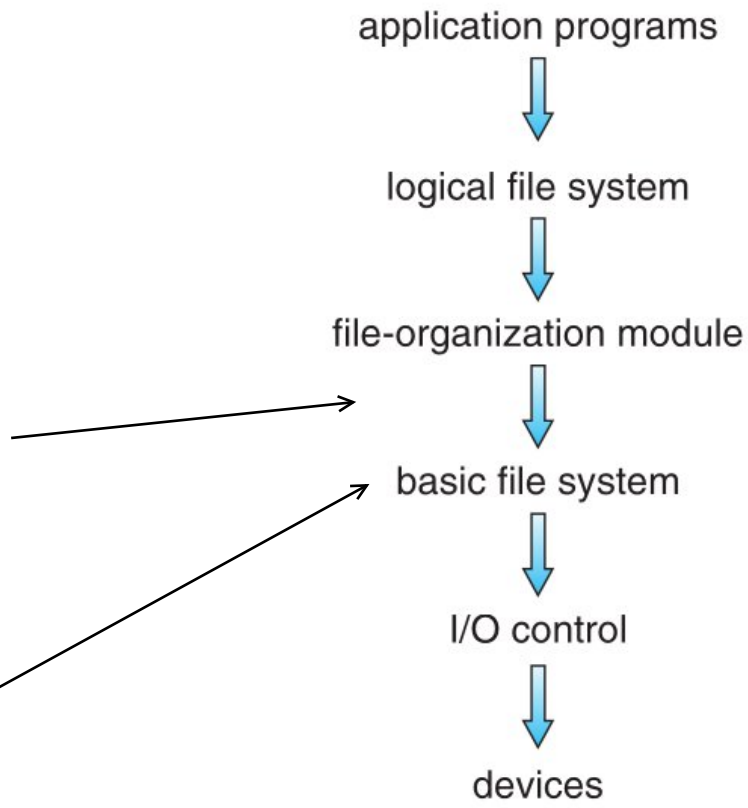


Fig 8. *Layered file system.*

Conception en couches du système de fichiers

```
// lire 100 octets de /home/ilias/notes.txt
int fd = open("notes.txt", O_RDWR);
read(fd, buf, 100);
```

Couche 5: CONTRÔLE DES E/S (PILOTE)

traduit en instructions pour le contrôleur du disque

(écrit les bons motifs de bits dans ses registres)

Couche 6: PÉRIPHÉRIQUE (DISQUE)

le bloc 4021 est lu, puis remonte jusqu'à buf

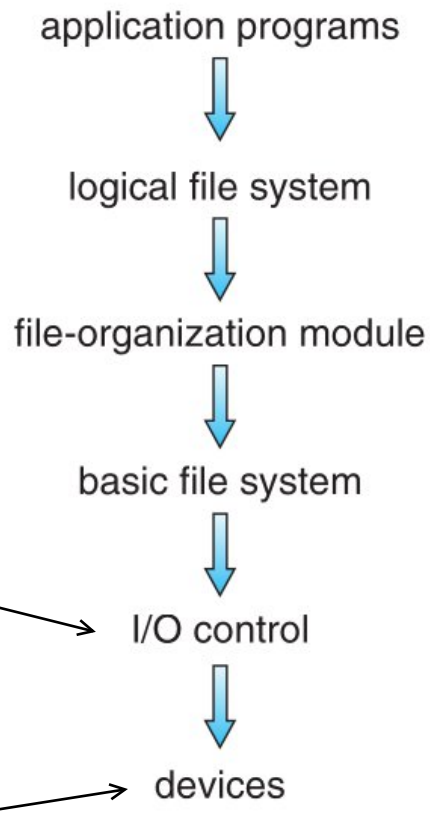


Fig 8. *Layered file system.*

Structure d’un système de fichiers

Les structures sur disque: Le système de fichiers peut contenir des informations sur la manière de démarrer un système d’exploitation qui y est stocké, le nombre total de blocs, le nombre et l’emplacement des blocs libres, la structure des répertoires, ainsi que les fichiers individuels.

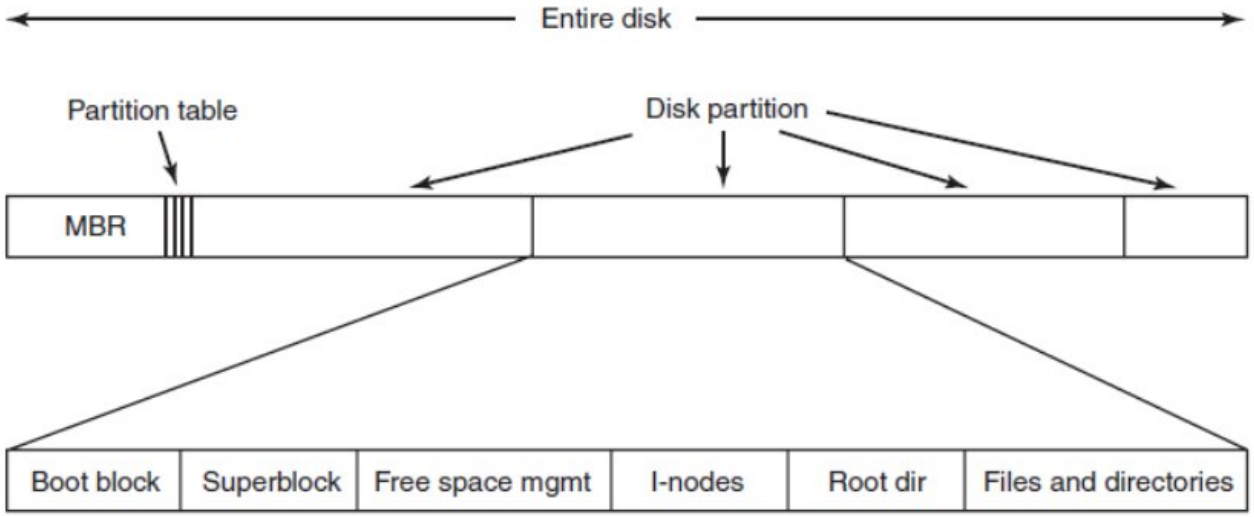


Fig 9. *File-System Layout*

Enregistrement de démarrage principal

MBR (Master Boot Record):

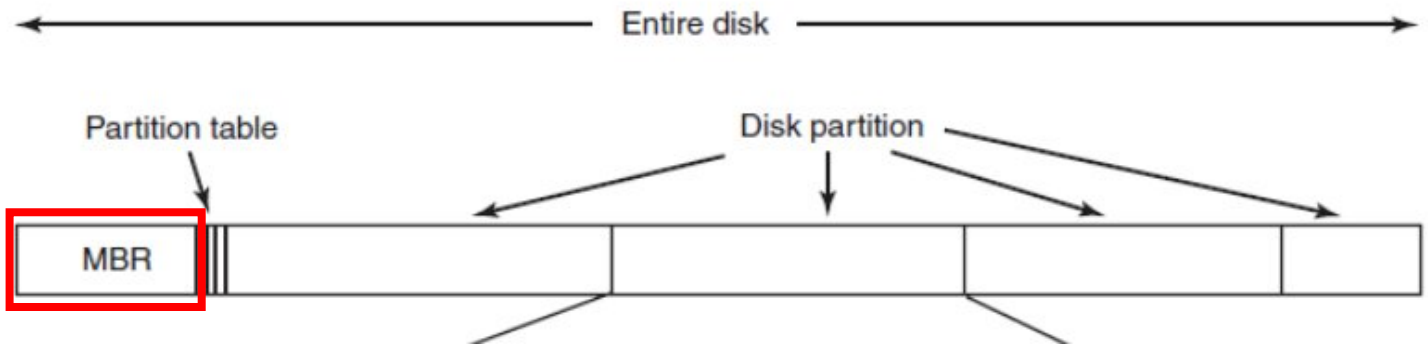


Fig 9a. *File-System Layout – Entire disk*

Le tout premier secteur du disque. Il contient un petit programme de démarrage exécuté par le BIOS au démarrage**. Il cherche la partition active et lui passe la main.**

Table de partitions

Partition table:

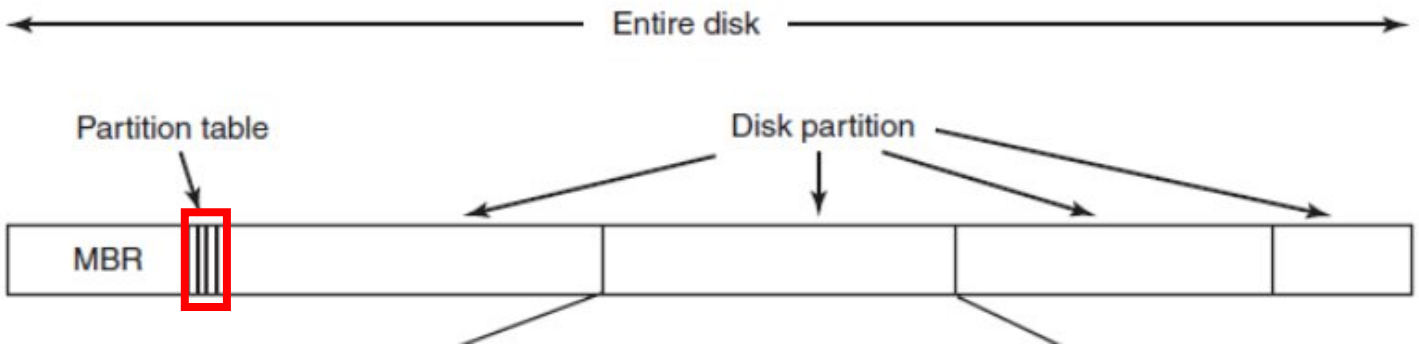


Fig 9a. *File-System Layout – Entire disk*

Un tableau qui indique, pour chaque partition, où elle commence, sa taille, et laquelle est active (celle qui contient l'OS à démarrer).

Partition du disque

Disk partition (les 4 zones)

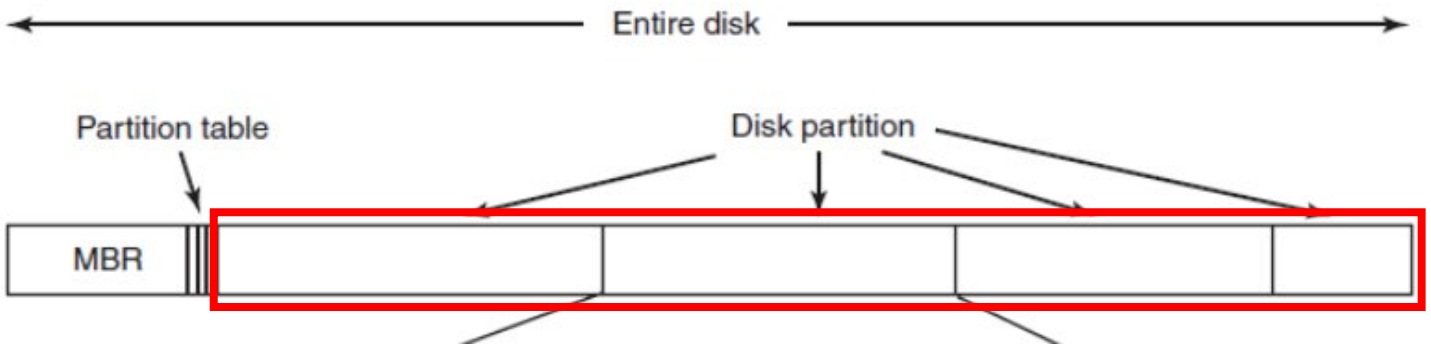


Fig 9a. *File-System Layout – Entire disk*

Une portion du disque traitée comme un disque indépendant. Chacune peut avoir son propre système de fichiers (ext4, NTFS, FAT…).

Bloc d'amorçage

Boot Block:

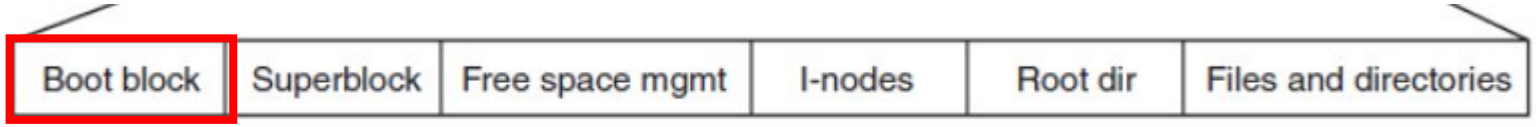


Fig 9a. *File-System Layout – Inside a partition*

Lors du démarrage de l'ordinateur, **le BIOS lit et exécute le MBR**. La première opération effectuée par le programme du **MBR consiste à localiser la partition active**, à en lire le premier bloc — appelé « bloc de démarrage » (boot block) — et à l'exécuter. Le programme contenu dans ce bloc de démarrage charge le système d'exploitation présent sur la partition. Chaque partition comporte un bloc de démarrage à son début, même si elle ne contient pas de système d'exploitation amorçable.

Le Super bloc

Superblock

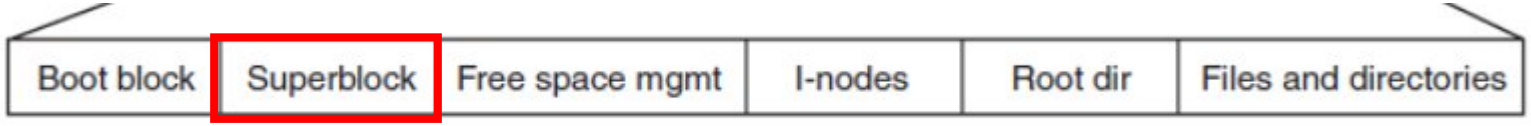


Fig 9a. *File-System Layout – Inside a partition*

Contient les informations globales de la partition : **nombre de blocs**, **taille des blocs**, **type de système de fichiers, compteurs de blocs et d'inodes libres.**

La gestion de l’espace libre

Free space mgmt



Fig 9a. *File-System Layout – Inside a partition*

La gestion de l’espace libre : une structure (un bit par bloc : 0 = libre, 1 = occupé) qui indique quels blocs sont disponibles.

L’I-nodes

I-nodes

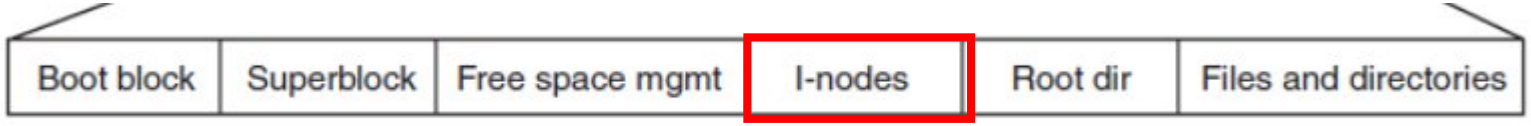


Fig 9a. *File-System Layout – Inside a partition*

Le tableau des inodes : une fiche par fichier (propriétaire, permissions, taille, pointeurs vers les blocs).

Le répertoire racine

Root dir

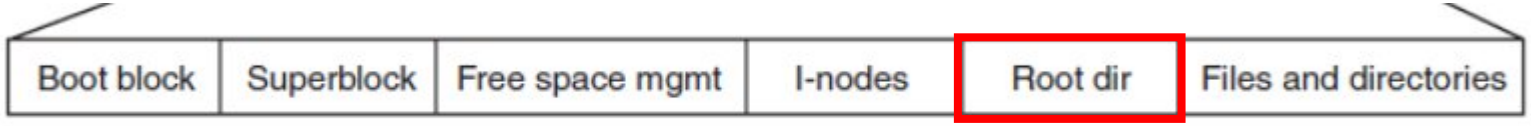


Fig 9a. *File-System Layout – Inside a partition*

Le répertoire racine / : le point de départ de tout chemin absolu.

Fichiers et répertoires

Files and directories

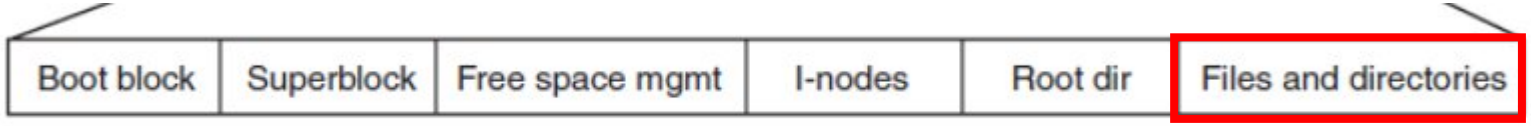


Fig 9a. *File-System Layout – Inside a partition*

Tout le reste de la partition : les blocs de données des fichiers et des autres répertoires.

L'inode : métadonnées d'un fichier

Inode (i-nœud) : structure par fichier stockant les métadonnées et les pointeurs vers les blocs de données.

- Il contient : type, permissions, propriétaire, taille, horodatages, compteur de liens, pointeurs de blocs
- Il ne contient pas le nom du fichier — le nom vit dans le répertoire
- Chaque fichier possède un inode, identifié par son numéro d'inode
- `ls -i` et `stat` affichent le numéro d'inode et ses métadonnées

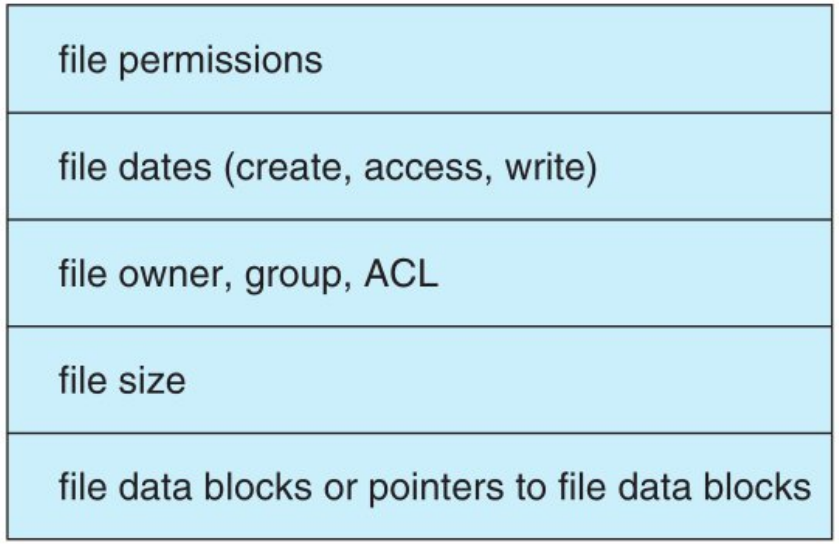
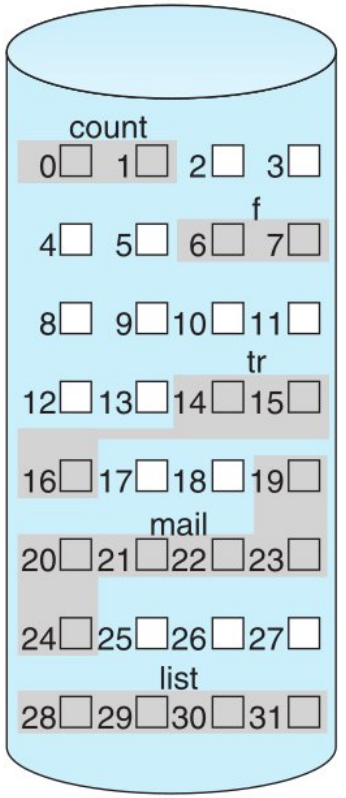


Fig 10. *A typical file-control block*

Allocation contiguë

Allocation contiguë : chaque fichier occupe une suite de blocs consécutifs sur le disque

- L'inode ne stocke que le bloc de départ et la longueur
- **Avantages** : accès séquentiel et direct très rapides, un seul positionnement de tête
- **Inconvénient majeur** : fragmentation externe — **l'espace libre se morcelle**
- Problème de l'allocation d'espace : quelle taille réserver à un fichier qui grandit ?
- Nécessite de connaître la taille à l'avance, ou de compacter périodiquement le disque



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Fig 11. *Contiguous allocation of disk space.*

Allocation chaînée et FAT

Allocation chaînée : chaque bloc contient un pointeur vers le bloc suivant du fichier

- L'inode ne stocke que le premier bloc ; la liste se parcourt de proche en proche
- Aucune fragmentation externe : chaque bloc libre est utilisable
- **Inconvénient** : l'accès direct est lent (parcours linéaire), et un pointeur perdu casse tout le fichier
- FAT (MS-DOS) : les pointeurs sont regroupés dans une table centrale, plus rapide à parcourir
- La FAT est chargée en mémoire, mais sa taille croît avec le disque

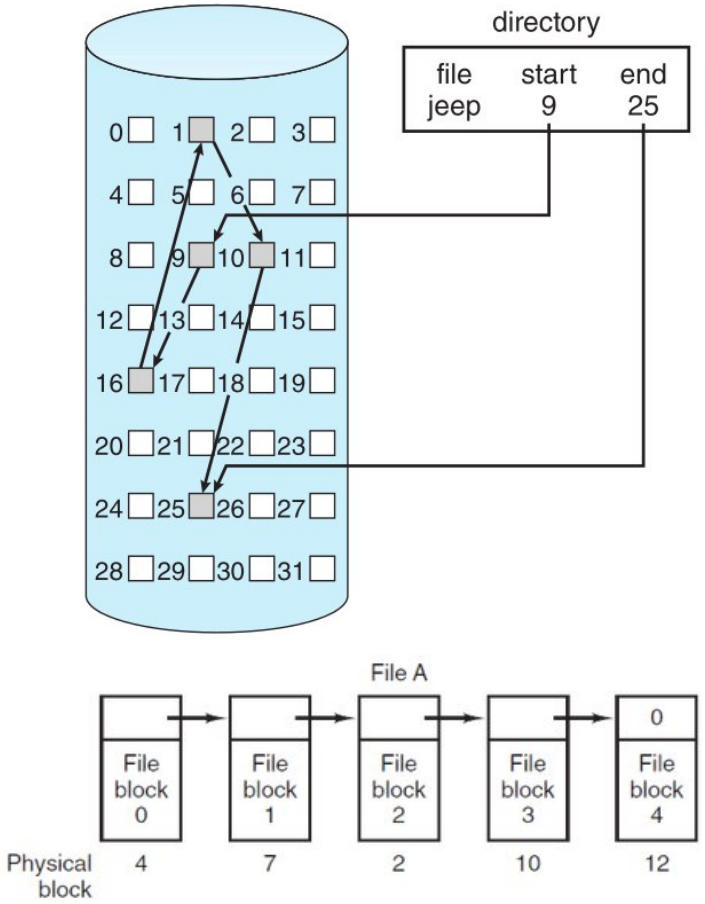


Fig 12. *Linked allocation of disk space*

Allocation indexée

Allocation indexée : tous les pointeurs du fichier sont regroupés dans un bloc d'index

- L'inode pointe vers ce bloc d'index, qui référence tous les blocs de données
- Accès direct efficace : on lit l'index, puis on saute au bon bloc
- Aucune fragmentation externe, et pas de chaînage fragile
- **Inconvénient** : surcoût de l'index (un bloc perdu par fichier), et l'index lui-même a une taille limitée
- C'est le principe de l'inode Unix (index multi-niveaux)

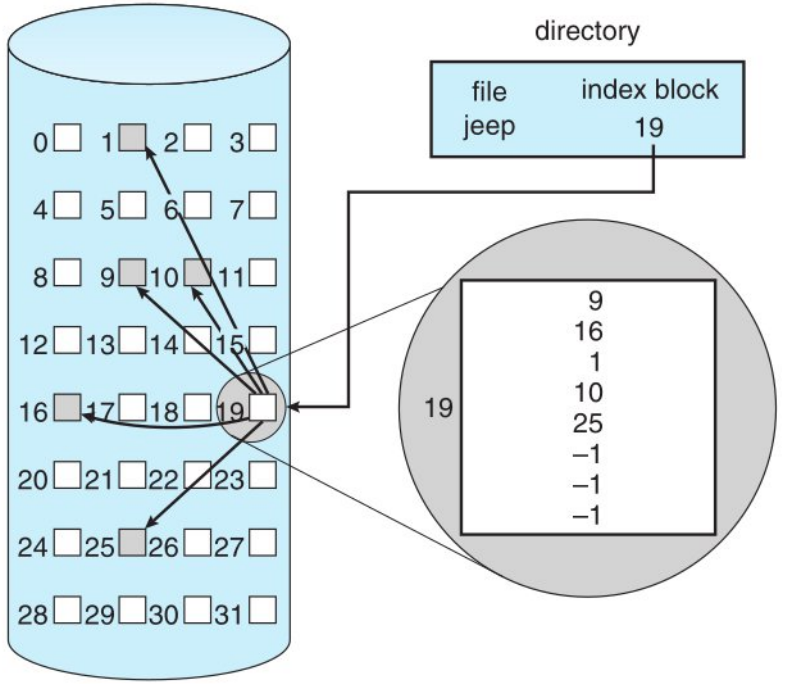


Fig 13. *Indexed allocation of disk space.*

L'inode Unix : pointeurs directs et indirects

- **Pointeurs directs** : référencent directement les premiers blocs du fichier (rapides pour les petits fichiers)
- **Indirection simple** : un bloc qui référence d'autres blocs de données
- **Indirection double** : un bloc référençant des blocs d'indirection simple
- **Indirection triple** : un bloc référençant des blocs d'indirection double
- Les petits fichiers n'utilisent que les pointeurs directs ; les gros fichiers utilisent les indirections

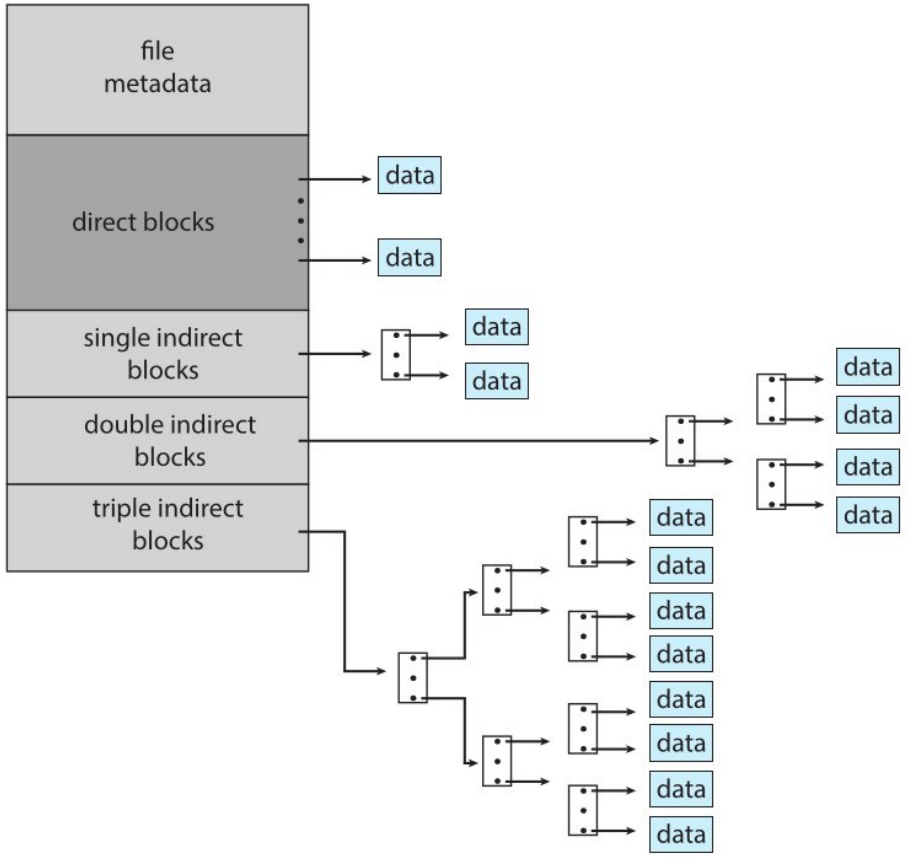


Fig 14. *The UNIX inode.*

Exemple : calcul de la taille maximale d'un fichier

Hypothèses : bloc de **4 Ko**, pointeur de **4 octets**, 12 pointeurs directs + 1 simple + 1 double + 1 triple

Étape 1 : combien de pointeurs par bloc ?

Un bloc fait **4 Ko** = **4096 octets**. Chaque pointeur (adresse d'un bloc) **occupe 4 octets**.

- **1024** pointeurs par bloc, donc un bloc de pointeurs peut désigner 1024 autres blocs.

Étape 2 : ce que chaque niveau permet d'adresser

Niveau	Chemin	Nombre de blocs de données	Calcul de la taille
12 directs	l'inode pointe directement vers les données	12	12 × 4 Ko = 48 Ko
Indirect simple	inode → 1 bloc de pointeurs → données	1024	1 024 × 4 Ko = 4 096 Ko = 4 Mo
Indirect double	inode → 1 bloc → 1 024 blocs de pointeurs → données	1 024 × 1 024 = 1 048 576	1 048 576 × 4 Ko = 4 194 304 Ko = 4 Go
Indirect triple	3 niveaux de blocs de pointeurs	$1\,024^3$ ≈ 1,07 milliard	4 × 1,07 milliard ≈ 4 294 967 296 Ko = 4 To

Section 5 : Liens physiques et symboliques

Lien physique → lien symbolique → comparaison → exemple pratique ln et ln -s

Le lien physique (hard link)

Lien physique : une entrée de répertoire supplémentaire pointant vers le même inode.

- `ln source.txt lien` crée un second nom pour le même fichier
- Les deux noms sont égaux : il n'y a ni « original » ni « copie », un seul inode et un seul contenu
- L'inode porte un **compteur de liens** (`ls -l` colonne `2`) : il s'incrémente à chaque lien
- Le fichier n'est supprimé que lorsque le compteur tombe à zéro (dernier nom retiré)
- Interdit sur les répertoires et impossible entre systèmes de fichiers différents

Le lien symbolique (symlink

Lien symbolique : un petit fichier spécial dont le contenu est un chemin vers un autre fichier.

- `ln -s cible lien` crée un lien symbolique nommé `lien` pointant vers `cible`
- C'est un fichier à part entière, avec son propre inode, marqué du type `l` dans `ls -l`
- Il peut **traverser les systèmes de fichiers** et pointer vers des répertoires
- Si la cible est supprimée, le lien devient cassé (dangling) : il pointe vers un chemin inexistant
- `readlink lien` affiche le chemin vers lequel pointe un lien symbolique

Le lien symbolique (symlink

Lien symbolique : un petit fichier spécial dont le contenu est un chemin vers un autre fichier.

- `ln -s cible lien` crée un lien symbolique nommé `lien` pointant vers `cible`
- C'est un fichier à part entière, avec son propre inode, marqué du type `l` dans `ls -l`
- Il peut **traverser les systèmes de fichiers** et pointer vers des répertoires
- Si la cible est supprimée, le lien devient cassé (dangling) : il pointe vers un chemin inexistant
- `readlink lien` affiche le chemin vers lequel pointe un lien symbolique

Comparaison : lien physique vs lien symbolique

- **Lien physique** : même inode que la cible ; **lien symbolique** : inode propre
- **Lien physique** : ne traverse pas les systèmes de fichiers ; **symbolique** : oui
- **Lien physique** : interdit sur les répertoires ; **symbolique** : autorisé
- **Lien physique** : la suppression de la cible laisse le lien intact ; **symbolique** : lien cassé
- **Lien physique** : deux noms égaux pour un seul fichier ; **symbolique** : un pointeur par chemin
- **Retenir** : le **lien physique** partage l'inode, le **lien symbolique** partage le chemin

Exemple pratique : ln et ln -s

- echo "contenu" > a.txt crée un fichier ; ls -li a.txt montre son numéro d'inode
- ln a.txt b.txt crée un lien physique : a.txt et b.txt ont **le même inode**
- ls -li confirme : numéro d'inode identique, compteur de liens à 2
- ln -s a.txt c.txt crée un lien symbolique : c.txt a un inode différent (***type l***)
- **Supprimer** a.txt : b.txt survit (**lien physique**), c.txt devient cassé (**symlink**)
- rm b.txt : le compteur de liens passe à 0 et le fichier est réellement supprimé

Quiz: Système de Fichiers



https://forms.cloud.microsoft/r/4MVFCuyes8